SOFTWARE DESIGN PROJECT

Dungeon Composite

A dungeon crawler built on four design patterns

Composite

Decorator

State

Singleton

Author	[Your name]
Course	[Course / module]
Supervisor	[Teacher name]
Date	[Date]

Language: Java 17 - Front-ends: console (Scanner) and 2D (Swing)

1. Project overview

Dungeon Composite is a small dungeon crawler. The player goes down **5 floors**. On each floor they pick **one room** out of two or three, fight or avoid the monsters inside, loot the treasure and take the stairs down. Some rooms hide a **vault** guarded by a boss. Every action can wake sleeping monsters, and awake monsters strike back. The goal is to reach the bottom alive with as much gold as possible.

The same game model is played through two interchangeable front-ends:

- **Console version** (`Main` / `GameManager.run()`) - numbered menus read with a `Scanner`.
- **2D version** (`Game2D`, `Sprites`) - a Swing window with pixel-art sprites. The hero moves with the arrow keys and walking into an element fights, loots or enters it.

Neither front-end contains game rules. Both only talk to the model through the `DungeonElement` interface and the `GameManager` singleton, which is what the design patterns make possible.

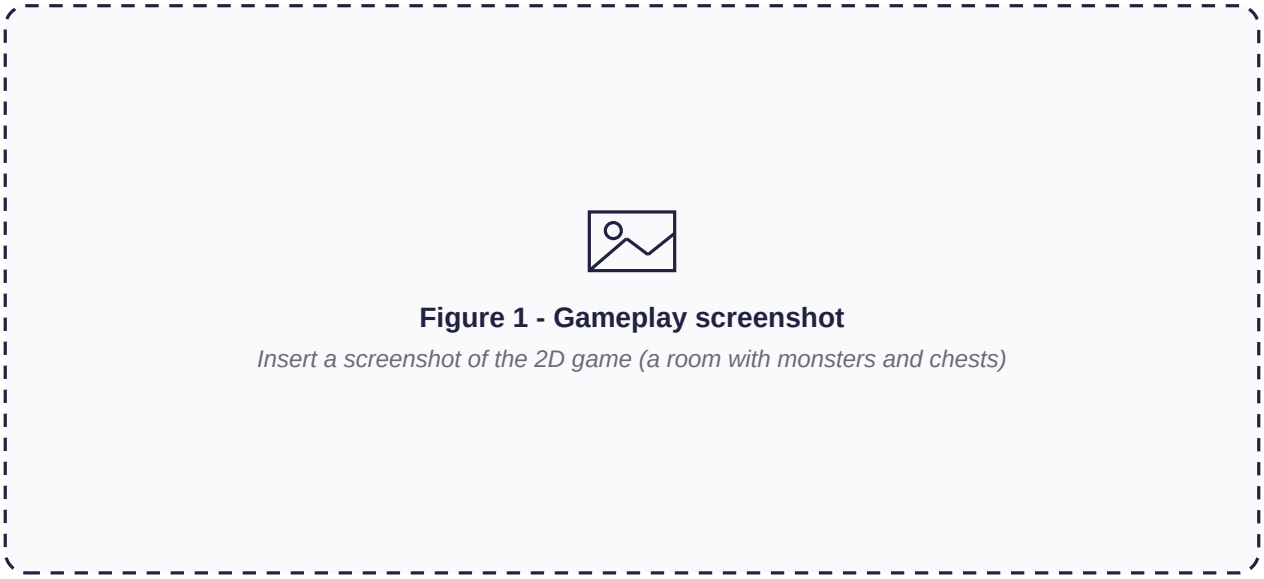


Figure 1 - Gameplay screenshot

Insert a screenshot of the 2D game (a room with monsters and chests)

2. Patterns at a glance

Pattern	Category	Main classes	Role in the game
Composite	Structural	DungeonElement, Room, Monster, Treasure	Dungeon -> floors -> rooms -> vaults, all treated the same way
Decorator	Structural	ElementDecorator, EliteMonster, BossMonster, CursedTreasure, EnchantedTreasure	Adds modifiers (x2, x3, curse, seal) on top of any element at runtime
State	Behavioural	MonsterState, AwakeState, SleepingState, DefeatedState	Monster behaviour (damage, bounty, label) changes with its mood
Singleton	Creational	GameManager	One game session: one dungeon, one player, one message log

3. Architecture

The class diagram below shows how the patterns fit together. `DungeonElement` is the centre of the design: it is the *Component* of the Composite and the type that every Decorator wraps. `Monster` delegates its behaviour to a `MonsterState`, and `GameManager` owns the root `Room`.

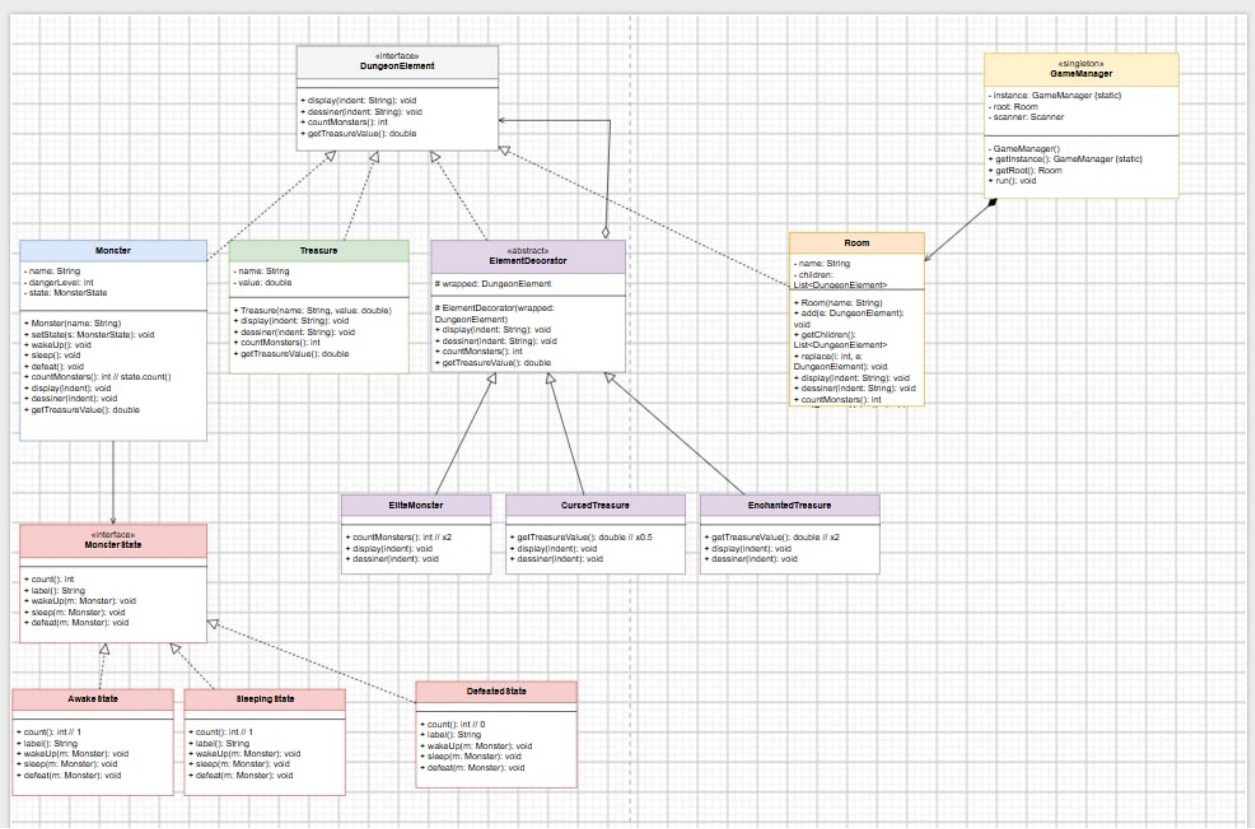


Figure 2 - UML class diagram of the project

Every element answers the same questions:

Method	Meaning in the game
describe()	Text shown to the player ("Elite Goblin (danger 2, sleeping)")
countMonsters()	How many living monsters the element contains
getTreasureValue()	Gold that can still be taken (bounty for monsters)
threat()	Damage dealt right now to whoever touches it
collect(player)	Takes the gold out and returns it (defeats monsters)
wakeUp() / sleep()	Noise propagation

PATTERN 1

Composite

Compose objects into tree structures and let clients treat single objects and groups uniformly.

Problem

A dungeon is nested: the dungeon has floors, floors have rooms, rooms contain monsters, treasures and sometimes a vault, which is itself a room. The game constantly needs totals such as "how much gold is in this room, vault included?" or "how many monsters are still alive?". Without a common type, every loop would need `instanceof` checks and special cases per level.

Solution in the project

Role	Class	Responsibility
Component	DungeonElement	Common interface for everything in the dungeon
Composite	Room	Holds a List<DungeonElement> and delegates every call to its children
Leaf	Monster	A single enemy; its numbers come from its state
Leaf	Treasure	A pile of gold that empties once collected

```
public class Room implements DungeonElement {
    private final List<DungeonElement> children = new ArrayList<>();

    public double getTreasureValue() {
        return children.stream().mapToDouble(DungeonElement::getTreasureValue).sum();
    }
    public int countMonsters() {
        return children.stream().mapToInt(DungeonElement::countMonsters).sum();
    }
    public void wakeUp() { children.forEach(DungeonElement::wakeUp); }
}
```

In the game

- The whole dungeon is **one tree**: `Dungeon -> Floor n -> Room nX -> Vault`, built in `buildDungeon()`.
- Totals are one call: `root.getTreasureValue()` gives the gold in the whole dungeon for the end-of-game score.
- Vaults are just Rooms inside Rooms. Entering one in 2D pushes it on a stack and the same drawing code renders it.
- `display(indent)` prints the recursive "what was left behind" report at the end of the console game.



Figure 3a - Game

A room containing a vault door



Figure 3b - Code

Room.java or buildDungeon()

PATTERN 2

Decorator

Attach additional responsibilities to an object dynamically, without subclassing it.

Problem

Monsters and treasures come in variants: elite, boss, cursed, enchanted. Subclassing every combination (EliteGoblin, CursedGem, BossEliteOrc...) would explode the class count. The variants also need to be chosen *randomly at generation time*, not fixed at compile time.

Solution in the project

`ElementDecorator` implements `DungeonElement`, holds a wrapped `DungeonElement` and forwards every call to it. Concrete decorators override only what they change:

Decorator	Changes	Effect for the player
EliteMonster	countMonsters x2, threat x2, bounty x2	Harder hit, double reward. 20% of monsters
BossMonster	threat x3, bounty x2, "BOSS" prefix	Vault guardian, the most dangerous enemy
CursedTreasure	after collect(), calls player.curse()	All later gold is halved. A risk/reward choice
EnchantedTreasure	value x2, but 0 while its boss is alive	Vault relic, sealed until the boss is defeated

```
public class EnchantedTreasure extends ElementDecorator {
    private final DungeonElement boss;

    private boolean sealed() { return boss.countMonsters() > 0; }

    @Override
    public double collect(Player player) {
        return sealed() ? 0 : wrapped.collect(player) * 2;
    }
}

// generation: decorators are applied at random, at runtime
room.add(random.nextDouble() < 0.2 ? new EliteMonster(m) : m);
```

In the game

- The game code never knows whether an element is decorated. `interact()` calls `threat()` and `collect()` and the decorators adjust the result.
- Decorators stack: `new BossMonster(new EliteMonster(m))` would give x6 threat with no new class.
- The 2D view shows decorations visually: a gold glow for Elite, horns for Boss, a purple chest for Cursed, chains on a sealed Enchanted chest.



Figure 4a - Game

A boss in a vault, or a cursed chest



Figure 4b - Code

ElementDecorator + one subclass

PATTERN 3

State

Let an object change its behaviour when its internal state changes, as if it changed class.

Problem

A monster behaves very differently depending on whether it is asleep, awake or dead: how much it hurts, whether it counts as alive, what bounty it gives and how it reacts to noise. Writing this as `if (state == AWAKE) ...` in every method scatters the rules.

Solution in the project

State	count()	attack()	wakeUp()	defeat()
SleepingState	1	0	-> Awake	-> Defeated
AwakeState	1	danger x 4	stays	-> Defeated
DefeatedState	0	0	stays	stays

```
public class Monster implements DungeonElement {
    private MonsterState state;
    public int threat()        { return state.attack(this); }
    public int countMonsters() { return state.count(); }
    public void wakeUp()       { state.wakeUp(this); }
}
public class SleepingState implements MonsterState {
    public int attack(Monster m) { return 0; }
    public void wakeUp(Monster m) { m.setState(new AwakeState()); }
}
```

In the game

- **Stealth gameplay**: sleeping monsters deal no damage, so the player can loot around them.
- **Noise**: after every action `makeNoise()` has a 35% chance to call `wakeUp()` on each element. The state decides whether that changes anything.
- **Counter-attack**: `strikeBack()` sums `threat()` and only awake monsters contribute.
- **Defeat**: `DefeatedState.count() = 0` unseals vault relics (EnchantedTreasure) automatically.
- **2D visuals** follow the state: red eyes and bouncing (awake), closed eyes and "zZ" (sleeping), a skull (defeated).



Figure 5a - Game

Sleeping vs awake monster, and a skull



Figure 5b - Code

MonsterState and its 3 states

PATTERN 4

Singleton

Ensure a class has only one instance and provide a global point of access to it.

Problem

A play session has exactly one dungeon, one player and one message log. Both front-ends (console and 2D) must work on that same session. If each created its own manager, they could end up with two different dungeons or two different players.

Solution in the project

```
public final class GameManager {
    private static GameManager instance;
    private final Player player = new Player();
    private final Room root;

    private GameManager() { root = buildDungeon(); }   // nobody else can call new

    public static GameManager getInstance() {
        if (instance == null) instance = new GameManager();
        return instance;
    }
}

// both front-ends
GameManager.getInstance().run();                       // Main (console)
private final GameManager game = GameManager.getInstance();  // Game2D
```

In the game

- The dungeon is generated **once**, in the private constructor, on first access.
- The game rules live in one place: `interact()`, `strikeBack()` and `makeNoise()` are shared by the console and 2D versions.
- The 2D window reads the player, the dungeon root and the message log through `getPlayer()`, `getRoot()` and `takeLog()`.

Limitation. The lazy initialisation is not thread-safe. That is fine here because the game runs on a single thread (the console thread or the Swing event thread). A multi-threaded version would use an eager instance or a holder class.



Figure 6a - Game

HUD: HP bar, gold, message log



Figure 6b - Code

GameManager.getInstance()

5. How the patterns work together

A single move in the 2D game goes through all four patterns. Suppose the hero walks into an **Elite Goblin** that is **awake**, in a room that also holds a vault:

#	What happens	Pattern
1	Game2D asks `GameManager.getInstance()` to `interact(target, items)`	Singleton
2	`target.threat()` - EliteMonster doubles what the wrapped Monster returns	Decorator
3	Monster delegates to `AwakeState.attack()` = danger x 4	State
4	`collect()` pays the bounty (x2 by the decorator) and switches the state to Defeated	State + Decorator
5	`strikeBack()` sums `threat()` over the room. Vaults are skipped as Rooms	Composite
6	`makeNoise()` calls `wakeUp()`. A sleeping monster changes state, a dead one ignores it	State
7	The view redraws by asking each element `countMonsters()`, `threat()` and `getTreasureValue()`	Composite

Benefits observed

- **Open for extension**: a new modifier (e.g. *PoisonedTreasure*) is one new decorator class, and nothing else changes.
- **A new monster mood** (e.g. *Enraged*) is one new MonsterState and needs no edits to Monster.
- **Two user interfaces, one model**: the 2D version was added without touching any pattern class.
- **No type checks for game rules**: totals and damage come from polymorphic calls on the tree.



Figure 7 - End screen

Insert a screenshot of "YOU ESCAPED!" or "YOU DIED" with the final gold

6. Additional content

Space reserved for the content you will add: personal analysis, difficulties encountered, test results, possible improvements, etc.

6.1 [Section title]

[Your text here]



Figure 8 - Screenshot

Game or code screenshot supporting this section

6.2 [Section title]

[Your text here]



Figure 9 - Screenshot

Game or code screenshot supporting this section

7. Conclusion

[Your conclusion here]